

AI Web Testing Platform

Product and technical specification

Version 0.1 for review

8 October 2026

Prepared for Muthuraman

The product we want to build

A subscription application that helps software teams understand their web application, create reviewable test scenarios, and run repeatable regression tests when they fix bugs or add features. Customers connect their own test environment, supply access and product requirements, and receive evidence of what passed, what failed, and what remains untested.

The first version focuses on browser based web applications. WhatsApp and Instagram represent the kind of complex customer product this platform could eventually support; the first release is not an integration that obtains their internal source code or tests every part of those public services.

The first useful outcome

A customer connects one supported test application, reviews generated scenarios, approves a repeatable suite, and runs it against a new build. The report identifies a deliberately introduced regression and links to the exact failed step, expected result, observed result, and execution evidence.

Confirmed decisions and proposed defaults

- Confirmed: start with web applications. Create and review this specification before application development begins.
- Proposed: begin with a hosted pilot, desktop Chromium, an accessible staging environment, and human review of generated tests. Add source repository integration after the core testing loop works.
- Proposed: use AI for discovery, scenario drafting, and failure explanation; use a controlled browser runner and explicit assertions to determine test outcomes.

All requirements, limits, technologies, and delivery stages below are proposals for review unless explicitly marked confirmed. This document authorizes no implementation or deployment by itself.

Reading guide

Pages 2 to 7 describe the product and customer experience. Pages 8 to 10 describe the proposed system, data, and isolation requirements. Pages 11 to 13 cover release criteria, delivery stages, decisions, and technical references.

1 Customers and release scope

Who uses the platform

The initial customer is a small or medium software team with a web application, a stable test environment, and a person who can confirm intended behavior. The buyer wants less repetitive manual regression work and faster feedback on releases.

Role	Main responsibility
Owner	Manage the workspace, members, connected environments, usage limits, and future subscription.
Maintainer	Provide requirements, review and approve scenarios, maintain test data, and configure runs.
Viewer	Read test cases, run results, and permitted evidence without changing test definitions.

Pilot scope

Included in the pilot	Deferred from the pilot
Customer owned web apps in desktop Chromium	Native mobile apps, desktop software, and device farms
URL, test accounts, text requirements, and guided discovery	Complete understanding from a URL alone
Login, navigation, forms, validation, and supported business flows	Unrestricted autonomous exploration or automatic bug fixing
Reviewed scenarios, manual runs, reports, and run history	GitHub automation, nightly scheduling, and billing checkout
Isolated workspaces, basic roles, quotas, and stored evidence	Enterprise SSO, private network runners, and multi region hosting

Supported connection contract

The customer supplies an authorized test URL, allowed domains and paths, dedicated accounts for relevant roles, disposable test data, and a short description of the main workflows. A reachable test environment is required for the hosted pilot. Source code is optional at this stage.

The pilot supports standard browser interactions, including text entry, selections, buttons, links, and file inputs with prepared fixtures. CAPTCHA, unattended MFA, canvas only interfaces, complex drag interactions, and unsupported external identity flows produce an explicit limitation or blocked step. These cases are not silently marked tested.

Full regression means all active approved cases in the selected environment and suite. It does not mean every possible input, role, browser, or behavior in the application has been tested.

2 Customer workflow

Connect the application

The customer creates a workspace and project, adds the staging URL, and configures approved navigation and API domains. They enter or upload product requirements and register test accounts. The connection check verifies reachability and the configured login flow, without treating login success as proof of application coverage.

Discover and build an application map

The customer selects starting pages and important journeys. A bounded browser session records reachable pages, navigation, controls, form fields, validation messages, and role specific differences. Guided journeys fill gaps that a crawler cannot reach. Discovery reports inaccessible pages and stops at its configured page, time, or cost limit.

The map groups evidence into modules such as Authentication, Profile, Feed, and Messaging. Each observation retains its environment, role, source page, capture time, and build identifier when available. Existing behavior is an observation until a reviewer confirms it as intended.

Generate and review scenarios

AI drafts happy paths, negative cases, boundary cases, and permission checks from the supplied requirements and discovered controls. The customer reviews steps, data, expected results, and evidence, then edits, rejects, or approves each case. Missing requirements become questions in the review screen.

Run and investigate

The customer selects a build and approved suite. The platform prepares isolated test data, executes the suite, and streams progress. A failure report shows the first relevant failed step and its evidence. A maintainer can rerun a case, export a bug report, or update a test through a new reviewed version.

Example for a social web application

A customer adds the requirement “Users can edit their own post, but cannot edit another user’s post.” The platform proposes cases for editing, saving, cancellation, missing content, and user ownership. Two dedicated test users and independently seeded posts are required.

After a developer changes the post editor, the regression suite checks that edits persist after reload and that the second user cannot modify the first user’s post through the tested UI. Server side authorization needs a separate API assertion before the report can claim that broader coverage. If the edit is not saved, the run fails with the observed text and screenshot.

Bug fixes and new features

For a bug fix, preserve a case that reproduces the reported problem and verify the fix before running the broader suite. For a new feature, create scenarios from its acceptance criteria and add approved cases to regression. Existing unrelated cases retain their approved expected results.

3 Functional requirements

The following requirements define the minimum pilot. Requirement identifiers remain stable as the specification evolves.

ID	Requirement and required behavior
FR01	Workspaces and access. Authenticate users, enforce Owner, Maintainer, and Viewer roles, and isolate every customer record and artifact.
FR02	Projects and environments. Store a project, staging URL, allowed domains, role accounts, test data policy, and environment readiness status.
FR03	Requirements. Accept plain text or Markdown requirements, version them, and link each scenario to its requirement or observed source.
FR04	Discovery. Explore approved navigation with configurable limits; save an application map and show unreachable, excluded, and unobserved areas.
FR05	Scenario drafting. Produce structured test steps, preconditions, data needs, expected results, priority, and a source for the expectation.
FR06	Review and versioning. Allow editing, approval, rejection, archiving, and approval withdrawal. Changes to steps, data, or assertions require renewed review.
FR07	Execution. Run selected or full approved suites asynchronously. Support cancellation, bounded retries, independent sessions, and per case outcomes.
FR08	Evidence. Preserve expected and actual values, timestamps, failed step, screenshots, and permitted trace data, with explicit artifact upload failures.
FR09	History and export. Compare runs, filter failures, link cases to requirements, and export scenario CSV and an HTML or JSON run report.
FR10	Usage and operations. Enforce time, concurrency, discovery, model usage, and storage quotas; record changes and show actionable job errors.

Requirements for the commercial release

CR01 adds a repository connection and verified build triggered runs. CR02 adds scheduled suites, impact based selection with a full suite fallback, and repository check results. CR03 adds subscriptions, entitlements, invoices through a payment provider, and metered usage. These are separate delivery gates, not completed pilot capabilities.

Handling unsupported work

The platform preserves the attempted action and explains what input or capability is missing. A scenario that cannot be compiled, lacks test data, or has no approved expected result remains outside the executable suite. Its exclusion is visible in coverage and run summaries.

4 Scenario model and AI behavior

A test case is a versioned contract

Each case contains a stable ID and version, module, title, priority, requirement references, environment constraints, user role, preconditions, test data references, ordered actions, expected results, cleanup steps, and approval history. Locators and assertions are stored as structured data; secret values are referenced, never embedded.

Example field	Example value
Case	POST 014 version 1 — Editing a post persists after reload
Source and role	Requirement REQ 008 version 2; authenticated post owner
Precondition	A unique post belongs to the test user; no other run uses it
Actions	Open post; choose Edit; replace text; Save; reload
Assertions	Updated text is visible after reload; old text is absent
Cleanup	Remove only the record created for this run
Approval	Maintainer approves the exact version before regression use

How AI is used

The model receives a limited, redacted set of relevant requirements and application observations. It proposes scenarios and explains source links and uncertainty. A schema validator checks required fields; a controlled compiler accepts only supported browser actions and assertion types. Invalid outputs are rejected with a visible reason.

Expected results must come from customer requirements, an approved baseline, or explicit reviewer confirmation. Observed behavior alone cannot establish correctness. Conflicting sources produce a review question. The system must not invent business rules, credentials, payment data, hidden APIs, or successful execution results.

Case lifecycle

Draft cases can become Ready for review, Approved, or Rejected. Approved cases can be Archived or flagged Needs review when their source or implementation changes. Approval binds the exact case version. Each run snapshots those versions; later edits cannot rewrite historical results.

A first execution validates whether the case is executable. A failed assertion is retained as a failure to investigate; the model cannot weaken it to obtain a passing baseline. Locator repair creates a proposed revision and preserves the original failed attempt.

Model availability and control

The AI provider and model remain configurable and unselected. Store provider, model, prompt version, source versions, and metered usage for each generation. If the service is unavailable or the budget is exhausted, allow manual cases and existing suite execution. Label any template based suggestions clearly; do not present them as an AI analysis.

5 Regression execution and change testing

Execution contract

A run records the selected environment, build identity, suite version, browser version, approved case versions, data setup, and retry policy before execution. Each case uses an isolated browser context and uniquely assigned fixtures. Cases sharing an account or record must be serialized unless isolation has been demonstrated.

The runner uses explicit assertions for visible text, field values, URLs, element state, and supported persisted outcomes. Playwright provides browser assertions that wait for the expected condition within a timeout. [1] Assertions verify the customer's approved behavior; an AI explanation never overrides their result.

Retries and interrupted work

Proposed default: one retry for a failed case only when its setup and cleanup can safely be repeated. Preserve every attempt. A retry pass is Flaky and remains visible; it is not a clean pass. A worker crash or environment outage is a runner error or blocked run, with infrastructure evidence separate from a product failure.

Cancellation stops new cases and requests worker termination. Record completed results and unfinished cases. A duplicate job or webhook must not create duplicate logical runs. Worker recovery must never blindly repeat an action that could have produced a side effect; mark the attempt interrupted when repeat safety is unknown.

Testing a bug fix

- Capture the bug description, expected behavior, affected role, and reproducible data as a reviewed case.
- When the previous build is available, show the case failing there and passing on the fixed build. Otherwise state that the historical reproduction is unverified.
- Run the approved regression suite against the fixed build and preserve unrelated failures separately.

Testing a new feature

Generate draft cases from new acceptance criteria, identify related existing modules, and execute the approved cases on the feature environment. A later repository integration can use changed files and requirement links to propose affected tests. Uncertain impact selection falls back to the full approved suite.

Build identity and release gates

In the pilot, a customer supplies a build label and the report marks whether it was verified. Automated repository runs require a deployment readiness signal and a verified association between commit, environment URL, and deployed build. Receiving a pull request event alone does not prove that a staging URL serves that commit.

Default automated gate: all required cases must pass without unresolved flaky, blocked, skipped, or errored outcomes. An empty or incomplete suite never produces a green release check. Full scheduled regression remains necessary even when fast impact based runs are introduced.

6 Screens and reporting

Screen	Primary customer actions
Workspace overview	See projects, latest runs, review backlog, environment readiness, and usage.
Connect project	Enter URL and requirements, configure test access, check connection, and start discovery.
Application map	Browse modules and roles; inspect discovered evidence, exclusions, and coverage gaps.
Scenario library	Search cases; inspect sources; edit steps and assertions; approve, reject, or archive.
Run setup and progress	Select build and suite; review data needs and estimated usage; run or cancel.
Run report	Inspect outcomes and attempts, compare history, view evidence, and export findings.
Project settings	Manage domains, role accounts, access, retention, limits, and future integrations.

Outcome language

Case outcome	Meaning
Passed	All required assertions passed on the initial attempt.
Failed	An executed assertion did not match its approved expectation.
Flaky	The initial attempt failed and an allowed retry passed.
Blocked	A prerequisite prevented execution, such as expired authentication.
Skipped	A configured rule intentionally excluded the case, with a recorded reason.
Errored or cancelled	Runner failure or requested cancellation prevented a complete result.

Evidence and coverage

A report includes case versions, environment, build verification, expected and actual values, failed step, duration, attempts, and available screenshots and traces. Playwright traces can show actions and snapshots for investigation. [2] Missing evidence is displayed as missing rather than replaced with generated content.

Show approved requirements with at least one linked active approved case divided by all in scope approved requirements. Label this “requirements with test cases,” not code coverage. Show separately which cases executed and which requirements had passing assertions on this build. Unknown and excluded requirements remain visible.

AI failure summaries identify likely causes as hypotheses and link to evidence. The dashboard shows real empty states before any run; sample data belongs in a clearly labeled demo project.

7 Proposed system architecture

Use a modular application service and separate disposable browser workers. This keeps the first implementation manageable while isolating untrusted customer pages and expensive execution from the customer dashboard.

Component	Proposed implementation and responsibility
Web dashboard	React and TypeScript for project setup, scenario review, progress, and evidence.
Application API	Node.js and TypeScript for authentication, authorization, requirements, approval, runs, and quotas.
Primary storage	PostgreSQL for tenant records, versions, run state, job idempotency, and audit events.
Job orchestration	Redis backed queue, with PostgreSQL as the durable authority for runs and recovery.
Discovery and execution	Playwright Test in isolated workers with fixed resource and network boundaries.
AI gateway	Provider adapter, source retrieval, redaction, schema checks, budget tracking, and prompt versioning.
Artifact storage	Private object storage for screenshots, traces, and reports; authorized expiring access.

How data moves

The dashboard sends a request to the API. The API checks tenant access and limits, writes a durable job record, and dispatches work through a queue. A worker obtains a short lived, job scoped credential, performs discovery or execution, uploads permitted evidence, and records progress. The API delivers updates to the dashboard.

Discovery observations and approved requirements feed the AI gateway. Validated scenario drafts return to the review screen. Only approved case versions enter the execution manifest. The result processor computes outcomes from runner output and builds the report; AI may add a separately labeled explanation.

Important boundaries

The pilot executes a restricted action schema, not arbitrary model generated JavaScript or shell commands. A browser session is not a sufficient tenant security boundary: worker containers or equivalent sandboxes also need network policy, short lived credentials, no host mounts, and per job resource limits.

The API handles multiple organizations from the start. Do not share browser profiles, secrets, model retrieval indexes, or object storage permissions across workspaces. Keep queue messages small and refer to stored records by tenant scoped identifiers.

Deployment choice

Start with one hosting region and managed database, queue, and object storage where practical. Select the cloud, AI provider, identity provider, and payment provider during review. Pin and verify dependency versions during implementation; this specification does not prescribe unverified version numbers.

8 Data and integration contracts

Core records

Record group	Required information
Organization and membership	Tenant ID, user identity, role, membership status, entitlement, and audit history.
Project and environment	Name, URL, allowed destinations, role accounts as secret references, readiness, and build reference.
Requirement and observation	Versioned content, module, provenance, timestamps, role, environment, and source build.
Scenario and approval	Stable case ID, immutable version, steps, assertions, data references, sources, reviewer, and approval time.
Suite and run	Selected versioned cases, trigger, build identity, state, timeouts, limits, and idempotency key.
Attempt and artifact	Case result, action log, expected and actual values, error classification, private artifact key, and retention expiry.
Usage and audit event	Organization, actor, operation, timestamp, units, reason, and correlation ID.

Proposed API surface

Versioned routes under `/api/v1` cover projects, environments, requirements, discoveries, scenarios, approvals, suites, runs, artifacts, and usage. Reads require tenant authorization. Creation returns a resource ID; long operations return an accepted job ID and a status URL. Errors use a stable code, readable message, and correlation ID.

Run creation and discovery requests accept idempotency keys. Case edits require a version check to prevent overwriting a reviewer's changes. Approval targets one immutable case version. Cancellation is a distinct action. Artifact downloads require authorization before issuing an expiring link.

Repository integration in the commercial release

A GitHub App can receive subscribed push or pull request events through webhooks. [4] Request only necessary repository permissions, validate delivery signatures, deduplicate deliveries, and support disconnection. Reading source requires an explicit customer connection and a defined repository scope.

A deployment event or customer CI callback supplies the target URL and commit identity. Verify the allowed destination and readiness before running. Publish results against the tested commit; do not label an unverified shared staging run as validation of a specific pull request.

Portable results

Case exports include stable identifiers, human readable steps, expected results, source links, and version. Run exports include a manifest and per attempt outcomes. Exported definitions contain secret references only. Evidence links remain permission controlled and expire independently of exported report files.

9 Access data handling and test isolation

Customer and environment boundaries

Enforce organization membership and role checks on every API, job, result, and artifact access. Verify tenant ownership after resolving an identifier. Test cross tenant access explicitly. Owners approve the target environment and action scope; authorized test writes may create, edit, submit, and remove only the configured fixtures.

Discovery defaults to approved navigation. HTTP method alone cannot establish whether an action is safe. Form submission, messaging, deletion, and external integrations require an explicit scenario and test data policy. Use test mailboxes and provider sandboxes for flows that send messages or create transactions.

Outbound network and browser isolation

Hosted workers enforce destination allowlists at the network boundary, including redirects, subresources, API calls, and WebSockets. Block localhost, private and reserved ranges, link local metadata endpoints, and unsafe schemes; validate resolved addresses and prevent DNS rebinding. Private customer environments require a future scoped runner, not an exception that exposes platform infrastructure.

Run workers without platform administrator credentials or host filesystem access. Enforce CPU, memory, time, download size, and request limits. Destroy temporary browser profiles and revoke job credentials on completion or termination.

Secrets and authentication

Encrypt credentials in a dedicated secret store; keep only secret references in application records and restrict access to the relevant execution job. Never include passwords, session cookies, authorization headers, or private keys in AI context or exports. Browser storage state can contain credentials that impersonate the test account and must be treated as a secret. [3] Expired access becomes a blocked connection requiring renewal.

AI and artifact data handling

Treat page content, repository files, and uploaded requirements as untrusted input, including instructions embedded inside them. They cannot change tool permissions, approve cases, or request secrets. Show what classes of customer data may reach the selected AI provider and obtain workspace configuration before enabling external inference.

Restrict input capture to test data; redact sensitive text and network fields before persistence where supported. Screenshots and traces can contain sensitive data that automated redaction misses. Allow sensitive journeys to disable these artifacts, record the evidence limitation, and keep retained raw artifacts private. Retention is configurable; the proposed default is 30 days for evidence and 180 days for run summaries.

Audit and deletion

Record changes to access, destinations, secret references, approvals, runs, and limits without recording secret values. Project deletion revokes access immediately and queues data and artifact removal with a visible status. Define backup expiry, provider retention terms, and recovery procedures before customer launch.

10 Pilot acceptance and quality targets

Use a controlled web application containing known good workflows and deliberately seeded defects. These tests are release gates for the platform, not a claim that arbitrary customer applications have complete coverage.

Gate	Acceptance condition
Connection	Connect a permitted test app, validate two role accounts, and show actionable errors for a wrong URL or expired session.
Discovery	Map the agreed demo journeys, preserve sources, respect exclusions and limits, and display unreachable areas.
Scenario review	Generate valid drafts; reject missing assertions; edit and approve a version; prevent an unapproved revision from running.
Real execution	Execute at least 20 approved cases across five agreed workflows. Every displayed result has a matching runner record.
Defect detection	Detect each seeded defect covered by an approved assertion; show the exact expected and actual values and available evidence.
Repeatability	Run an unchanged, stable fixture suite five times. Investigate inconsistent outcomes and expose every retry as a separate attempt.
Failure handling	Verify timeout, cancellation, worker loss, duplicate request, unavailable AI, artifact upload failure, and cleanup failure behavior.
Isolation	Verify cross tenant access denial, secret redaction, destination blocking, and independent browser and test data sessions.
Honest reporting	An empty, blocked, partial, or flaky required suite cannot appear as a clean regression pass. Coverage shows its denominator.

Proposed initial limits and service targets

Defaults for the pilot: two concurrent browser jobs per workspace; 50 pages or 15 minutes per discovery; 100 cases per suite; two minutes per case; and a 60 minute run deadline. The deadline includes setup and retries and may stop a large suite before completion; the result must show all unexecuted cases.

Measure routine metadata API latency with a target below one second at the 95th percentile under agreed pilot load. Publish run progress within five seconds of receiving a worker event, and stop launching cases immediately after accepted cancellation. Target worker shutdown within 30 seconds. These are engineering targets to validate, not contractual service commitments.

Success measures

Track time from connected environment to first approved passing suite, reviewer acceptance and edit rates, flaky case rate, customer confirmed defect detections, and cost per completed run. Capture AI usage and browser minutes separately. Set commercial targets after pilot evidence; do not promise a fixed percentage of bugs found.

11 Delivery plan and commercial model

Development starts only after the owner reviews this specification and confirms the initial scope. Sequence work by demonstrated outcomes; assign calendar estimates after team size, hosting choices, and target application are known.

Stage	Deliverable and exit condition
A Foundation	Workspace access, tenant isolation, project connection, test credentials, and a controlled demo app. Exit: two isolated workspaces can connect and inspect their own projects.
B Discovery and review	Bounded discovery, requirement intake, application map, AI scenario drafts, and versioned approval. Exit: a reviewer can trace every approved expectation to a source.
C Regression pilot	Isolated execution, data setup and cleanup, evidence, outcomes, history, export, limits, and cancellation. Exit: all pilot acceptance gates on page 11 pass.
D Commercial release	Repository and deployment connections, scheduling, verified build checks, subscription checkout, entitlements, metering, and operational support. Exit: end to end billing and run controls pass in provider sandboxes.
E Broader coverage	Additional browsers, deeper API checks, visual comparison, private runners, and enterprise controls. Prioritize from pilot evidence. Native mobile requires a separate scope.

How customers buy and use it

The proposed model is a hosted workspace subscription with an included execution allowance. Candidate usage dimensions are browser execution minutes, AI generation usage, concurrency, and retained evidence. Avoid pricing only by test case count: cases have different execution costs and can be split or duplicated.

The pilot uses manually provisioned workspaces and explicit quotas. Self service purchase, plan upgrades, cancellation, and invoices belong to the commercial release. The entitlement service controls new work when a plan or quota changes and preserves access to existing results according to the agreed retention policy.

Cost visibility and limits

Before a run, show an estimate based on case history and selected concurrency, clearly labeled as an estimate. Apply configured execution and AI budgets even if a job misbehaves. Report actual consumption afterward. Decide paid overage behavior before checkout is released; no surprise unlimited usage.

Main delivery risks

The largest uncertainties are inaccessible workflows, unclear expected results, changing test data, unstable environments, and generated tests that require substantial review. Guided setup, explicit assertions, immutable case versions, isolated fixtures, and a small pilot cohort are the proposed responses. Measure each risk before widening scope.

12 Review decisions and references

Decisions to confirm before development

Decision	Proposed starting point
First customer application	One owned test app with five important workflows and two user roles.
Delivery scope	Stages A to C first; add commercial automation and billing after pilot validation.
Hosting and access	Hosted service in one region; publicly reachable staging URLs with dedicated test accounts.
Technology	TypeScript application, PostgreSQL, a job queue, private object storage, and Playwright workers.
AI configuration	Select provider and model after comparing scenario quality, data terms, latency, and measured cost.
Review ownership	A named maintainer approves expected results and test revisions for each customer project.
Commercial constraints	Confirm budget, team capacity, retention, subscription allowances, and whether private runners are an early requirement.

Terms used in this specification

Scenario: a proposed sequence of actions and expected results. Assertion: an executable check of an expected result. Regression suite: approved cases rerun after a change. Flaky: a case with inconsistent attempts. Application map: the observed pages, controls, roles, and relationships. Coverage: a named, measurable set of requirements or cases, never an unspecified promise to test everything.

Technical references

Official documentation reviewed on 8 October 2026. The references support the underlying tool capabilities; platform behavior, scope, defaults, and release targets in this document are proposed product decisions.

[1] Browser assertions and waiting for expected conditions. Used in the execution proposal on page 6.

[Playwright assertions](#)

[2] Recorded actions, snapshots, and failure investigation. Used in the evidence proposal on page 7.

[Playwright Trace Viewer](#)

[3] Reusable authentication state and its sensitivity. Used in the access controls on page 10.

[Playwright authentication](#)

[4] Event subscriptions and webhook setup for repository integrations. Used in the integration proposal on page 9.

[GitHub Apps and webhooks](#)